API Parameters & Controlling Output

Fine-Tuning LLM Behavior

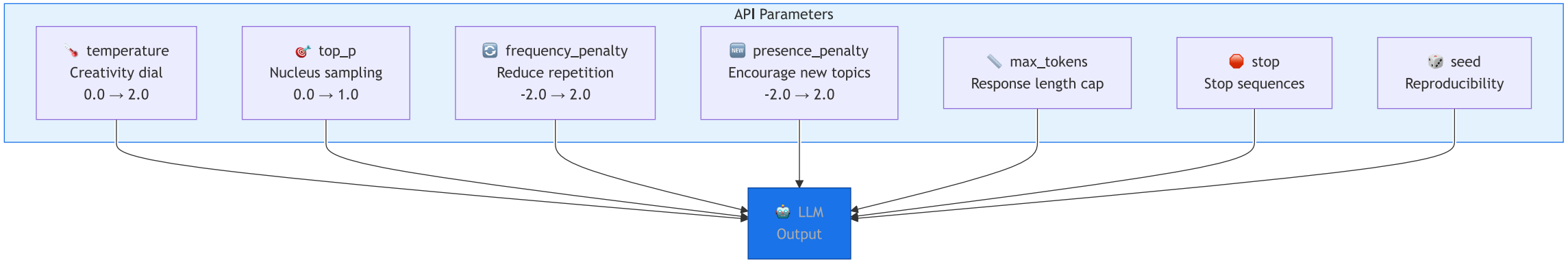
Day 2 - Session 3

KavinSchool

Session 3: What We'll Cover

1. `temperature`: the creativity dial
2. `top_p`: nucleus sampling
3. `frequency_penalty` & `presence_penalty`
4. `max_tokens`: response length control
5. `stop` sequences: clean termination
6. `seed`: reproducibility
7. Streaming responses for better UX

All the Knobs

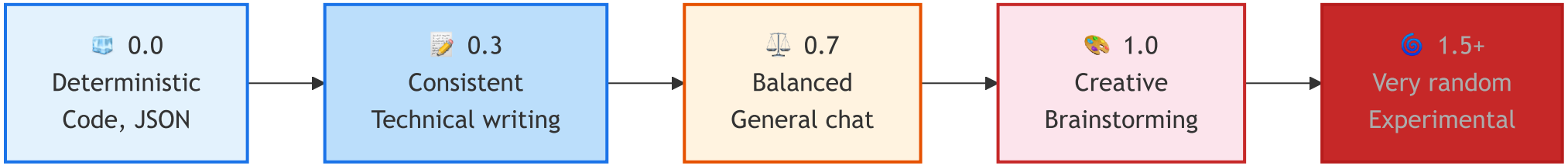


Temperature

The Creativity Dial

KavinSchool

Temperature: How It Works

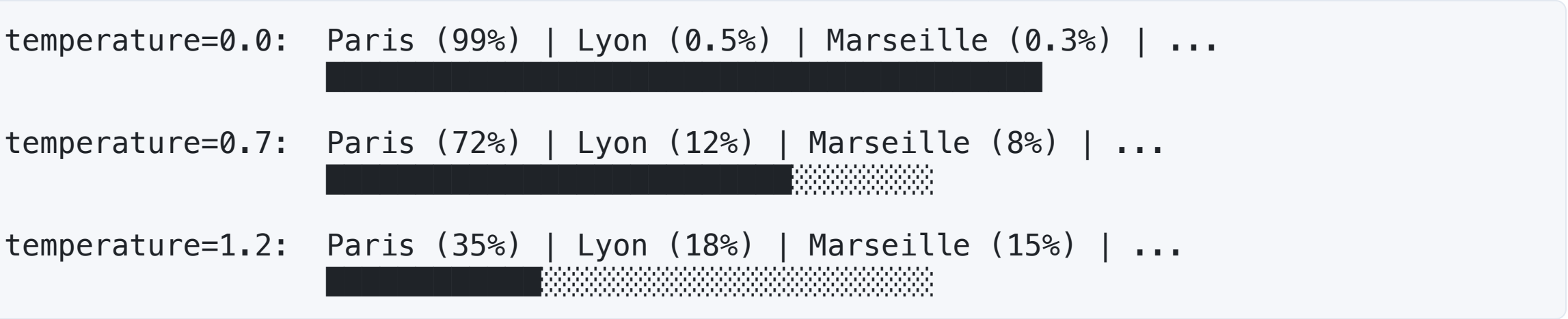


Temperature controls the **probability distribution** over next tokens:

Temperature	Distribution	Effect
0.0	Sharp peak on top token	Always picks the most likely word
0.7	Spread across likely tokens	Balanced between predictable and creative
1.0	Even wider spread	More surprising word choices
1.5+	Nearly flat	Random, often incoherent

Temperature: Visual Intuition

Next token after "The capital of France is":



> At 0.0, the model always says "Paris". At 1.2, it might say "Lyon", creative but potentially wrong.

Temperature: Use Case Guide

Task	Recommended	Why
Code generation	0.0	Syntax errors at higher temps
JSON extraction	0.0	Must be valid and consistent
Classification	0.0	Want the same label every time
Technical writing	0.2-0.3	Slight variation, mostly consistent
Summaries	0.3-0.5	Some flexibility in wording
General chat	0.7	Natural-sounding responses
Creative writing	0.8-1.0	Want variety and surprise
Brainstorming	1.0-1.2	Maximize divergent ideas

```
# Code generation: deterministic
response = get_completion(messages, temperature=0.0)

# Brainstorming: creative
```

top_p (Nucleus Sampling)

An Alternative to Temperature

KavinSchool

top_p: How It Differs from Temperature

`top_p` limits the model to the **smallest set of tokens** whose cumulative probability reaches p.

```
All tokens (sorted by probability):
Paris (45%) → Lyon (15%) → Marseille (12%) → Nice (8%) → Bordeaux (5%) → ...

top_p=0.6:  Consider only {Paris, Lyon}            (45+15 = 60%)
top_p=0.8:  Consider only {Paris, Lyon, Marseille} (45+15+12 = 72% ≈ 80%)
top_p=1.0:  Consider all tokens (default)
```

Parameter	Controls	Analogy
`temperature`	How spread the probabilities are	Blurring the focus
`top_p`	How many candidates to consider	Narrowing the candidates

> **OpenAI recommendation:** Change **one** of temperature or top_p, not both.

top_p: Practical Guidance

```
# Conservative: only top tokens
response = get_completion(messages, temperature=1.0, top_p=0.3)

# Default behavior
response = get_completion(messages, temperature=0.7, top_p=1.0)
```

When to use top_p over temperature:

- When you want creative output but need to avoid truly bizarre tokens
- `temperature=1.0, top_p=0.9`: creative within reason
- `temperature=0.0, top_p=1.0`: deterministic (temperature wins)

> In practice, most developers just use `temperature` and leave `top_p=1.0`.

Frequency & Presence Penalties

Reducing Repetition, Encouraging Variety

KavinSchool

frequency_penalty & presence_penalty

Both reduce repetition, but in different ways:

Parameter	Range	How It Works
`frequency_penalty`	-2.0 to 2.0	Penalizes tokens **proportional** to how often they appeared
`presence_penalty`	-2.0 to 2.0	Penalizes tokens that appeared **at all** (binary)

```
# Reduce word repetition in long-form text
response = get_completion(
    messages=messages,
    frequency_penalty=0.5,  # penalizes repeated words
    presence_penalty=0.3,   # encourages new topics
)
```

Frequency vs Presence: When to Use

Scenario	Setting
Code generation	Both at **0.0** (repetition is often correct in code)
Blog post writing	`frequency_penalty=0.3` to reduce word repetition
Brainstorming	`presence_penalty=0.5` to encourage new ideas
Poetry	`frequency_penalty=0.5, presence_penalty=0.5`
Data extraction	Both at **0.0** (don't penalize repeated structure)
Long conversations	`presence_penalty=0.3` to prevent topic loops

> **Default:** Both are 0.0. Only change them if you notice repetition problems.

max_tokens, stop & seed

Controlling Length, Termination & Reproducibility

KavinSchool

max_tokens: Response Length Cap

`max_tokens` sets the **maximum** number of tokens the model can generate.

```
# Short response
response = get_completion(messages, max_tokens=50)

# Long response
response = get_completion(messages, max_tokens=4096)
```

max_tokens: Response Length Cap

Situation	Recommended max_tokens
Classification (one word)	10
Short answer	100-200
Code snippet	500-1000
Detailed explanation	1000-2000
Long-form content	4096+

> **Warning:** If the response is cut off, `finish_reason` will be `"length"` instead of `"stop"`. Always check!

stop: Clean Termination

`stop` sequences tell the model to **stop generating** when it outputs a specific string.

```
# Stop after generating a complete function
response = litellm.completion(
    model=get_model(),
    messages=[{"role": "user", "content": "Write a Python function..."}],
    stop=["\ndef ", "\nclass ", "\n#"],  # stop before next function/class
)

# Stop after a specific delimiter
response = litellm.completion(
    model=get_model(),
    messages=[{"role": "user", "content": "Generate SQL..."}],
    stop=[";\n"],  # stop after the semicolon
)
```

> Useful when you want **exactly one** function, SQL query, or code block.

seed: Reproducibility

`seed` makes responses **more reproducible** (not perfectly deterministic):

```
# Two calls with same seed → same output (mostly)
response1 = get_completion(messages, temperature=0.7, seed=42)
response2 = get_completion(messages, temperature=0.7, seed=42)
# response1 ≈ response2 (highly likely to be identical)
```

When to use:

- **Automated testing:** consistent outputs for assertions
- **Demos:** reproduce the same response for presentations
- **Evaluation:** compare prompt changes with same randomness

> **Note:** `seed` + `temperature=0.0` gives the most deterministic results. But even with seed, OpenAI says results may vary due to infrastructure changes.

Streaming Responses

Better UX with Real-Time Output

KavinSchool

Streaming: Why It Matters

Without streaming: User waits 3-10 seconds for the complete response, staring at a blank screen.

With streaming: Tokens appear as they're generated, like someone typing in real-time.

```
Without streaming:                With streaming:
[waiting... 5 sec...]             P...
[waiting...]                      Py...
[waiting...]                      Python is...
[full response appears]           Python is a versatile...
```

> Every major chatbot (ChatGPT, Claude, Gemini) uses streaming. Your apps should too.

Streaming: Implementation

```
import litellm
from shared import get_model

response = litellm.completion(
    model=get_model(),
    messages=[{"role": "user", "content": "Explain decorators"}],
    stream=True,
)

full_response = ""
for chunk in response:
    delta = chunk.choices[0].delta.content
    if delta:
        print(delta, end="", flush=True)
        full_response += delta

print()  # newline at the end
```

Streaming: Key Details

Aspect	Non-Streaming	Streaming
Return type	Complete response object	Iterator of chunks
Time to first token	Full generation time	~200ms
Usage/tokens	In response object	In final chunk (or separate call)
Error handling	One try/except	Need to handle mid-stream errors
Memory	Full response in memory	Incremental

```
# Getting token usage from streaming (final chunk)
for chunk in response:
    if chunk.choices[0].finish_reason == "stop":
        # Some providers include usage in the last chunk
        if hasattr(chunk, 'usage') and chunk.usage:
            print(f"Tokens used: {chunk.usage.total_tokens}")
```

Session 3: Key Takeaways

1. **Temperature is the most important parameter.** 0.0 for code/JSON, 0.7 for chat
2. **top_p is an alternative to temperature.** Use one or the other, not both
3. **Penalties reduce repetition.** Use frequency_penalty for long text
4. **max_tokens prevents runaway costs.** Always set a reasonable limit
5. **stop sequences give clean termination.** Great for code generation
6. **seed improves reproducibility.** Essential for testing and demos
7. **Streaming is a must for UX.** Users expect real-time token display

> Next: Session 4, Prompt Iteration & Refinement!

Questions?

Session 3 Complete. Break Time!

KavinSchool